

QUINN CH. 8 NOTES - IMC08 Track A

1. CORE OPERATION

- $y = Ax$: $M \times N$ matrix times length- N vector gives length- M output
- Each $y[i] = \text{dot product of row } i \text{ of } A \text{ with } x$
- Sequential complexity: $2MN$ FLOPs
- Arithmetic Intensity ≈ 0.25 FLOP/byte (FP64, large square)
- $AI = 2MN / ((MN + N + M) \times 8)$

2. WHY MEMORY BANDWIDTH BITES

- $AI \approx 0.25$ FLOP/byte is below T4 ridge (0.79 F/B) $\rightarrow$ bandwidth-bound
- Adding more cores gives zero benefit — they wait idle for DRAM
- GEMM (matrix-matrix) has $AI = O(N) \rightarrow$ becomes compute-bound for large N
- Ch. 8 focuses on memory access patterns, not algorithmic tricks

3. PCAM DECOMPOSITIONS

Row-wise (1D):

- Partition contiguous row blocks to each process
- Comm: broadcast x to all (MPI_Bcast), no reduce needed
- Best for tall-skinny A ; simple and efficient

Column-wise (1D):

- Partition column blocks; compute partial y sums
- Comm: MPI_Allreduce on partial y — more expensive
- Best for short-fat A

2D Checkerboard:

- $p \times q$ process grid, each holds submatrix block of A
- Phase 1: broadcast x slices within column; Phase 2: reduce y within row
- Better scalability for large square matrices; comm = $O(M/\sqrt{p})$
- Requires Cartesian communicator in MPI

4. COMPLEXITY & SCALABILITY

- Parallel compute: $2MN/p$ per process
- Comm overhead: $O(N)$ for broadcast
- Efficiency degrades when $\text{comm} \approx \text{compute}$, i.e. $p \approx 2M/N$
- SpMV: same PCAM but load imbalance from variable nnz/row

5. SPARSE FORMATS

- CSR: values + col_indices + row_ptr; natural row partition; irregular
- ELL: pads all rows to same nnz; GPU-friendly; wastes mem if nnz varies
- HYB: ELL for regular rows, COO overflow; best of both worlds
- Rule: uniform nnz $\rightarrow$ ELL/HYB; irregular $\rightarrow$ CSR adaptive

6. GPU SPECIFICS

- Warp (32 threads) handles one row of GEMV
- Coalescing: row-major + row partition $\rightarrow$ stride-1 access $\rightarrow$ fully coalesced
- x-vector fits in L2 cache for moderate N $\rightarrow$ reduces DRAM pressure
- Shared memory tiling: load TILE of x to shmem $\rightarrow$ reuse across threads
- SpMV: warp divergence from unequal nnz/row is key inefficiency

7. LIKELY DEMO QUESTIONS

- Why BW-bound? $AI < \text{ridge point}$; FP units idle
- Why CSR not ELL? Random sparsity = unequal row lengths
- CUDA-aware MPI benefit? Eliminates GPU $\rightarrow$ CPU $\rightarrow$ NIC copy
- Why GPU SpMV less efficient than GEMV? Irregular col access, divergence
- Ridge point meaning? $\text{Peak FLOP/s} \div \text{Peak BW}$
- How does batching RHS help? GEMM AI grows as $O(K) \rightarrow$ compute-bound